DEVELOPER ARCHITECTURE MANUAL

SAVIT Chatbot System & EssemChat Platform

Comprehensive Codebase Guide & Onboarding Playbook

Target Audience: Backend & Fullstack Engineers
Architecture Pattern: Monolithic Laravel + Inertia.js React SPA
Core Domains: E-commerce, WhatsApp AI Bot, Tenant OS

Repository: `SAVIT_CHAT_BOT_SYSTEM`
Generated: July 2026
Version: 1.0 Production

Table of Contents

1. Executive Summary & Platform Overview

The **EssemChat / SAVIT Chatbot System** is an enterprise multi-tenant e-commerce platform integrated with an automated WhatsApp sales AI agent, storefront management, cart/order recovery system, and multi-gateway payment processing.

> 💡 **Architectural Philosophy**
>
> Rather than splitting frontend and backend into detached microservices (which increases hosting complexity on standard cPanel environments), the codebase employs a **Monolithic Single-Domain Architecture**. Laravel handles data, authentication, API routes, and Inertia server bridges, while React handles dynamic client-side rendering seamlessly without requiring an explicit external REST client build step.

Core Capabilities:

- **Multi-Tenant E-Commerce:** Each tenant (`Company`) gets isolated products, settings, orders, payment gateways, and WhatsApp configurations.
- **AI Cognitive Engine:** Powered by `UnifiedCompanyBrainService` and `ReplyGuardService` , executing tool calls (order processing, catalog queries, cart recovery) over WhatsApp.
- **Integrated Payment Gateways:** Native integrations for **Paystack**, **Flutterwave**, **Pesapal**, and manual cash delivery.
- **Single-Host Deployment:** Tailored for deployment on cPanel or Linux VPS under a single document root (`public/`).

2. Repository Layout & Key File Locations

Directory / File Path	Role & Purpose
`LARAVEL_BACKEND/app/Http/Controllers/`	API & Web Controllers partitioned into `Api/Admin`, `Api/Company`, and `Web`.
`LARAVEL_BACKEND/app/Models/`	Eloquent Database Models (`Company`, `Product`, `Order`, `PaymentGateway`, etc.).
`LARAVEL_BACKEND/app/Services/Agent/Brain/`	AI Cognitive Brain (`UnifiedCompanyBrainService.php`) & tool dispatch system.
`LARAVEL_BACKEND/app/Services/`	Domain business logic (`OrderPaymentService`, `PaystackService`, `StorefrontService`).
`LARAVEL_BACKEND/resources/js/Pages/`	Inertia React pages (`admin/`, `dashboard/`, `store/`, `pay/`).
`LARAVEL_BACKEND/resources/js/lib/`	API action helpers (`api-actions.ts`) and SWR fetch hooks (`api-hooks.ts`).
`LARAVEL_BACKEND/routes/`	Laravel route definitions: `api.php`, `web.php`, `console.php`.
`scripts/pack-cpanel.py`	Automated script to build assets and generate production deployment `EssemChat-cPanel.zip`.

3. Monolithic Architecture & Data Model

3.1 Multi-Tenant Isolation Pattern

Every tenant is represented by a `Company` record. Data models like `Product`, `Order`, `CompanySetting`, and `PaymentGateway` maintain a `company_id` foreign key.

```
// Example: Tenant Scoped Query in Controllers
$companyId = $request->user()->company_id;
$orders = Order::where('company_id', $companyId)
    ->with(['items', 'paymentGateway'])
    ->latest()
    ->paginate(15);
```

3.2 Core Entity Map

- `Company`: Tenant profile, domain name, currency, and AI features.
- `User`: System users linked to a company (Roles: `admin`, `owner`, `agent`).
- `Product`: E-commerce inventory item (price, stock, images, category).
- `Order`: Customer purchase containing line items, payment status, shipping details, and recovery status.
- `PaymentGateway`: Tenant-specific credentials (API keys, webhooks) for payment providers.
- `Conversation` & `Message`: Log of customer communications via WhatsApp bot.

4. The AI Agent Engine & Cognitive Brain

The AI component is located under `LARAVEL_BACKEND/app/Services/Agent/Brain/UnifiedCompanyBrainService.php`.

🤖 Agent Message Lifecycle

1. **Incoming Meta Webhook**
   WhatsApp message arrives at `Api/WhatsAppWebhookController`.
2. **Cognitive Context Building**
   `UnifiedCompanyBrainService` retrieves conversation history, company catalog context, and active cart state.
3. **Tool Execution & Intent Match**

The Agent executes tool functions (e.g. `ProcessOrderMessageTool`, checking product availability, or generating payment links).

4. **ReplyGuard Safety Check**
   `ReplyGuardService` verifies AI response against business constraints before dispatching to Meta API.

5. Frontend Architecture (Inertia.js + React)

The application uses **Inertia.js** to bridge Laravel controllers directly with React components. Controllers return Inertia responses instead of raw blade templates or pure JSON:

```
// Example Laravel Inertia Controller Response
return Inertia::render('dashboard/settings/page', [
    'settings' => $company->settings,
    'gateways' => $company->paymentGateways,
]);
```

React Component Directory (`resources/js/Pages/`)

- `admin/` — Platform super-admin dashboard (tenant metrics, global settings).
- `dashboard/` — Tenant seller management portal (inventory, settings, orders, AI setup).
- `store/` — Customer-facing storefront and product catalog pages.
- `pay/` — Secure checkout & payment gateway redirect page.

6. Step-by-Step Developer Workflows

Scenario A: Adding a New Field or Setting to Tenant Dashboard

1. **Database Migration**
   Create migration in `database/migrations/` if adding column to DB.

2. **Update Eloquent Model**
   Add attribute to `$fillable` in `app/Models/CompanySetting.php`.

3. **Update Controller & API Helper**
   Update `SettingsController.php` and add frontend mutation function in `resources/js/lib/api-actions.ts`.

4. **Update React Interface**
   Modify `resources/js/Pages/dashboard/settings/page.tsx` to include input control.

Scenario B: Building & Packaging for Production

Always use the dedicated packaging script rather than manual zipping to guarantee clean builds without dev artifacts:

```
# Run from repository root
python3 scripts/pack-cpanel.py
```

> **Packaging Script Output:** Generates `EssemChat-cPanel.zip` at repository root containing optimized autoloader dependencies, compiled Vite assets in `public/build/`, and excluded dev modules.

7. Production Build, Deployment & Operations

Environment / Target	Command / Procedure
Local Development	Run `php artisan serve` in terminal 1 & `npm run dev` in terminal 2 inside `LARAVEL_BACKEND/`.
Automated Packaging	Run `python3 scripts/pack-cpanel.py` from repo root.

Environment / Target	Command / Procedure
cPanel Upload	Upload `EssemChat-cPanel.zip` to cPanel File Manager, extract, set document root to `public/`.
Database Migrations	Run `php artisan migrate --force` in cPanel Terminal or via cron command.

8. Developer Golden Rules & Best Practices

1. Never Swallow Errors or Return Silent Dummy Fallbacks: Always trace root causes in log tracebacks when debugging failures.

2. Preserve API Contracts: When changing model attributes or function signatures, inspect every invocation across both frontend (`api-actions.ts`) and backend controllers.

3. Verification Command Requirement: Never declare work finished without executing build checks (`npm run build`) or unit tests.